FECAP - FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO

Curso de Ciência da Computação
6º Semestre - Turma 6º CCOMP

SMASH

PLANO DE RESPOSTAS AOS RISCOS

Entrega 2 - Redes de Computadores e Cibersegurança

Projeto 7 - Grupo Smash

Felipe Vallim Soares - 24026060
Guilhermy Mariano Lisboa Garcia - 23025371
Gustavo Oliveira Demetrio - 24026213
Saulo Pereira de Jesus - 24026095

Prof. Victor Bruno Alexander Rosetti

São Paulo
2026

IDENTIFICAÇÃO DO DOCUMENTO

Quadro 1 - Identificação do documento.

Campo	Informação
Projeto	Smash - parede instrumentada de treino de tênis (Projeto 7)
Disciplina	Redes de Computadores e Cibersegurança - Entrega 2
Data	06/11/2026
Versão do documento	2.0 (a versão 1.0 é o Plano de Gerenciamento de Riscos da Entrega 1)
Documento anterior	Plano de Gerenciamento de Riscos, versão 1.0 (Entrega 1, 25/09/2026)
Repositório	*https://github.com/2026-2-NCC6/Projeto7*
Código de referência	Tag *entrega2-ciberseguranca-aikido*, pasta *src/Entrega1*. A Entrega 2 continuou o código da Entrega 1, na mesma pasta; a tag marca o estado entregue agora, e os caminhos citados são relativos a essa pasta
Código-fonte	Acompanha este plano. Também pode ser baixado em *https://github.com/2026-2-NCC6/Projeto7/archive/refs/tags/entrega2-ciberseguranca-aikido.zip*

SUMÁRIO

1 INTRODUÇÃO....3
2 TABELA DE RESPOSTAS AOS RISCOS....4
3 DETALHAMENTO DAS ESTRATÉGIAS DE RESPOSTA....7
4 VARREDURA DO AIKIDO E CORREÇÃO DOS ACHADOS....8
5 MITIGAÇÕES IMPLEMENTADAS NO CÓDIGO....11
6 MONITORAMENTO E REVISÃO....12
APROVAÇÃO....13
ANEXO A - REGISTRO DE RISCOS ATUALIZADO....14
ANEXO B - TESTES DE SEGURANÇA....16
ANEXO C - CAPTURAS DO AIKIDO....17

1 INTRODUÇÃO

Este plano atualiza o Plano de Gerenciamento de Riscos da Entrega 1, que levantou 19 riscos no código e no projeto do Smash (R01 a R19) e definiu, para cada um, uma resposta e um prazo. Aqui estão a resposta dada a cada risco, o que foi corrigido no código e como cada correção foi testada. Os riscos novos começam em R20.

Nesta entrega, o professor pediu a análise do repositório no Aikido Security e o relatório gerado pela ferramenta. O Aikido apontou 8 achados, e a equipe conferiu cada um no código: três foram corrigidos e os outros cinco foram justificados (seção 4). O Security Audit Report do Aikido acompanha este plano em arquivo separado.

As correções também foram verificadas com testes automatizados, ataques simulados contra a API e a parede, npm audit, gitleaks, Semgrep e OWASP ZAP (seção 5 e Anexo B).

1.1 O que mudou no projeto desde a Entrega 1

O hardware mudou durante o desenvolvimento, e algumas descrições do Plano da Entrega 1 ficaram desatualizadas. O Quadro 2 registra as mudanças que afetam a leitura dos riscos.

Quadro 2 - Mudanças no projeto entre as entregas.

Aspecto	Entrega 1	Entrega 2
Parede	Nove alvos em matriz 3 × 3	Seis alvos em grade 3 × 2
Leitura dos piezos	Multiplexador de canais	Cada piezo no seu próprio pino ADC1 do ESP32
Integração MQTT	Prevista (R13)	Fora do escopo. O enunciado cita MQTT só como uma das opções de comunicação; o Smash usa WebSocket entre parede e aplicativo e HTTP entre aplicativo e API
Atuador	Não havia	LED da placa (GPIO 2), acionado pela parede e pelo aplicativo
Registro da sessão	O aplicativo enviava o resultado pronto	O servidor abre a sessão antes do jogo e aceita um único resultado por sessão (R04)

1.2 Resumo executivo

Dos 30 riscos registrados (19 da Entrega 1 e 11 novos), 13 estão concluídos, 5 têm a correção pronta no código e só dependem de publicar a API ou gravar a placa, 2 foram encerrados, 5 seguem em andamento, 4 foram aceitos com justificativa e 1 ainda não começou. Na Entrega 1 havia 3 riscos Extremos e 10

Altos; depois dos passos que faltam, nenhum fica Extremo e só o R14, que é físico, continua Alto.

2 TABELA DE RESPOSTAS AOS RISCOS

2.1 Resumo do status

O Quadro 3 resume a situação dos 19 riscos da Entrega 1. Na Entrega 1 havia 13 riscos Extremos ou Altos. Deles, 5 estão concluídos (R04, R06, R09, R11 e R17); 5 estão com a correção pronta no código e só falta publicar a API ou gravar a placa (R01, R02, R03, R05 e R07); 1 foi encerrado (R13, porque o MQTT saiu do projeto); e 2 continuam em andamento fora do código (R14, que é físico, e R15, reformulado para o prazo da apresentação).

Quadro 3 - Situação dos riscos da Entrega 1.

Status	Riscos	Total
Concluído	R04, R06, R08, R09, R10, R11, R12 e R17	8
Concluído no código; falta publicar a API ou gravar a placa	R01 (publicar) e R02, R03, R05 e R07 (gravar)	5
Encerrado	R13 (risco eliminado) e R19 (concluído desde a Entrega 1)	2
Em andamento, fora do código	R14, R15 (reformulado), R16 e R18	4

2.2 Tabela de respostas aos riscos

O Quadro 4 reúne a resposta dada a cada risco. "Nível" é a severidade na Entrega 1 ou, nos riscos novos, a classificação feita nesta entrega. Os responsáveis são as trilhas do Plano da Entrega 1. Nos prazos, "Apresentação" é a Semana FECAP de Tecnologia, de 17 a 24/11/2026. A severidade residual de cada risco está no Anexo A.

Quadro 4 - Plano de respostas aos riscos.

Risco	Nível	Estratégia	Ações aplicadas	Responsável	Prazo	Status
R01 - API em HTTP sem criptografia	Extremo	Mitigar	HTTPS com certificado, HTTP recusado e aplicativo sem http:// fora do desenvolvimento (Quadro 6).	Backend e Aplicativo	Apresentação	Concluído no código; falta publicar
R02 - WebSocket aceitava qualquer cliente	Extremo	Mitigar	Segredo de pareamento, comparado sem parar no primeiro caractere diferente, e impactos só para o aplicativo pareado (Quadro 6).	Firmware e Aplicativo	Apresentação	Concluído no código; falta gravar a placa
R03 - Senha do Wi-Fi previsível e pública	Extremo	Prevenir	Senha e segredo no *secrets.h*, fora do Git; sem ele o firmware não compila (Quadro 6).	Firmware	Apresentação	Concluído no código; falta gravar a placa

Risco	Nível	Estratégia	Ações aplicadas	Responsável	Prazo	Status
R04 - Resultado aceito sem prova	Alto	Mitigar	Sessão aberta pelo servidor, ticket assinado, limites de duração, batidas e pontos e um resultado por ticket (Quadro 6).	Backend	Entrega 2	Concluído
R05 - Qualquer cliente trocava a sessão da parede	Alto	Mitigar	Um aplicativo por vez; só o pareado manda comandos (Quadro 6).	Firmware	Apresentação	Concluído no código; falta gravar a placa
R06 - Login e cadastro sem limite de tentativas	Alto	Mitigar	Limite por IP e por conta, com bloqueio e resposta 429 (Quadro 6).	Backend	Entrega 2	Concluído
R07 - Parede sem limite de clientes e comandos	Alto	Mitigar	Limites de cliente, de tamanho e de comandos por segundo; ping desconecta quem sumiu (Quadro 6).	Firmware	Apresentação	Concluído no código; falta gravar a placa
R08 - CORS aberto e API em todas as interfaces	Médio	Mitigar	CORS desligado por padrão e escuta em 127.0.0.1 (Quadro 6).	Backend	Entrega 2	Concluído
R09 - Senha padrão do banco como reserva	Alto	Prevenir	Sem valor padrão: a API não sobe sem usuário e senha (Quadro 6).	Backend	Entrega 2	Concluído
R10 - Token de sete dias sem revogação	Médio	Mitigar	Validade de 12 horas, versão de token por usuário e público próprio (Quadro 6).	Backend	Entrega 2	Concluído
R11 - Dependências com alertas conhecidos	Alto	Mitigar	NestJS 11, multer fixado, correções compatíveis e alertas não alcançáveis justificados por escrito (Quadros 5 e 6).	Backend e Aplicativo	Entrega 2	Concluído
R12 - Pouca trilha de auditoria	Médio	Mitigar	Log em JSON de autenticação e recusas, sem senha nem token (Quadro 6).	Backend	Entrega 2	Concluído
R13 - MQTT sem controles definidos	Alto	Prevenir	MQTT retirado do escopo: o risco deixou de existir.	Firmware e Backend	-	Encerrado
R14 - Isolamento mecânico insuficiente	Alto	Mitigar	Validar em etapas (um alvo, dois lado a lado, os seis) e calibrar o limiar de cada alvo.	Firmware	Apresentação	Em andamento
R15 - Tempo para concluir os passos pendentes	Alto	Mitigar	Reformulado: revisão semanal dos passos da seção 6.1, com os riscos Extremos primeiro.	Gerente de Projeto	Apresentação	Em andamento
R16 - Alteração de critérios ou datas	Médio	Aceitar (ativo)	Relatórios, plano e código no mesmo repositório; datas e formatos confirmados com os professores.	Gerente de Projeto	Contínuo	Em andamento
R17 - Dados pessoais reais em demonstrações	Alto	Mitigar	Comandos para criar e apagar contas fictícias (Quadro 6).	Backend e Aplicativo	Entrega 2	Concluído

Risco	Nível	Estratégia	Ações aplicadas	Responsável	Prazo	Status
R18 - Única placa ESP32	Médio	Aceitar (ativo)	Modo simulado do aplicativo e eco Serial como contingência; avaliar uma segunda placa.	Firmware	Contínuo	Em andamento
R19 - Rede do local de demonstração	Médio	Prevenir	A parede cria a própria rede (SoftAP).	Firmware	-	Encerrado
R20 - Segredo de pareamento no pacote do aplicativo	Baixo	Aceitar (ativo)	Segredo diferente para cada parede, trocado antes de cada apresentação. Ele só serve dentro da rede WPA2 da parede.	Firmware e Aplicativo	Apresentação	Em andamento
R21 - Versões antigas deixam de salvar sessões	Médio	Mitigar	Usar só a versão atual do aplicativo nas demonstrações. O servidor responde com uma mensagem clara: "A sessão precisa ser iniciada antes de ser registrada."	Aplicativo	Entrega 2	Concluído
R22 - Limite de tentativas atrás de proxy	Médio	Prevenir	Variável *TRUST_PROXY=true* no ambiente publicado, para o IP de cada aparelho vir do proxy. Já implementada e documentada no *.env.example*.	Backend	Entrega 2	Concluído
R23 - API sem cabeçalhos de segurança	Muito Baixo	Mitigar	Cabeçalhos de segurança com o helmet e *Cache-Control: no-store* em todas as respostas (Quadro 6).	Backend	Entrega 2	Concluído
R24 - API aceitava o segredo JWT de exemplo	Médio	Prevenir	A API não inicia com o segredo de exemplo nem com um segredo curto (Quadro 6).	Backend	Entrega 2	Concluído
R25 - Bloqueio do login de outra pessoa	Médio	Aceitar (ativo)	É o custo conhecido de limitar tentativas por conta. O bloqueio é curto (15 minutos) e o limite por endereço impede que alguém bloqueie muitas contas de uma vez.	Backend	-	Aceito
R26 - Segredo de pareamento no endereço da conexão	Baixo	Aceitar (ativo)	A parede não registra o endereço pedido e só é alcançável dentro da própria rede WPA2. Trocar o segredo junto com a senha do Wi-Fi.	Firmware	-	Aceito
R27 - Cadastro revela se o e-mail tem conta	Médio	Aceitar (ativo)	O aviso ajuda quem já tem conta, e o limite de tentativas do R06 impede testar muitos e-mails em pouco tempo. O login não revela se o e-mail existe: a mensagem de erro e o bloqueio são iguais para e-mail com e sem conta (Anexo B).	Backend	-	Aceito
R28 - Sem exclusão da conta pelo jogador	Médio	Mitigar	Criar a exclusão da conta pelo próprio jogador. Até lá, a equipe apaga os dados a pedido.	Backend e Aplicativo	Apresentação	Não iniciado

Risco	Nível	Estratégia	Ações aplicadas	Responsável	Prazo	Status
R29 - Comparação do segredo no emulador	Muito Baixo	Mitigar	Comparação com *timingSafeEqual*, como o firmware já fazia (Quadro 6).	Firmware	Entrega 2	Concluído
R30 - GitHub sem lista de IPs permitidos	Baixo	Aceitar (ativo)	Recurso exclusivo do GitHub Enterprise Cloud. Usar autenticação em dois fatores nas contas com acesso de escrita (Quadro 5).	Gerente de Projeto	Contínuo	Aceito

3 DETALHAMENTO DAS ESTRATÉGIAS DE RESPOSTA

As estratégias são as definidas no Plano da Entrega 1. Esta seção mostra como cada uma foi usada; a resposta de cada risco está no Quadro 4.

3.1 Estratégias para ameaças (riscos negativos)

- **Prevenir:** eliminar a causa do risco. Usada em R03 (senha fora do repositório), R09 (sem senha padrão), R13 (MQTT fora do escopo), R19 (rede própria da parede), R22 e R24 (segredo de exemplo recusado).
- **Mitigar:** reduzir a probabilidade ou o impacto até um nível aceitável. Usada em R01, R02, R04 a R08, R10 a R12, R14, R15, R17, R21, R23, R28 e R29.
- **Transferir:** não foi usada: nenhum risco foi repassado a terceiros. A opção pode aparecer no R01, se a API for publicada em um provedor que cuide do certificado HTTPS.
- **Aceitar (ativo):** manter o risco, com justificativa escrita e contingência. Usada em R16, R18, R20, R25, R26, R27 e R30. Dentro do R11, os alertas não alcançáveis foram aceitos da mesma forma, como o Plano previa.

3.2 Estratégias para oportunidades (riscos positivos)

Nenhuma oportunidade foi registrada neste ciclo. Como ganho da entrega, o script *npm run security:test* passa a repetir os ataques a cada mudança e também poderá ser apontado para a placa real.

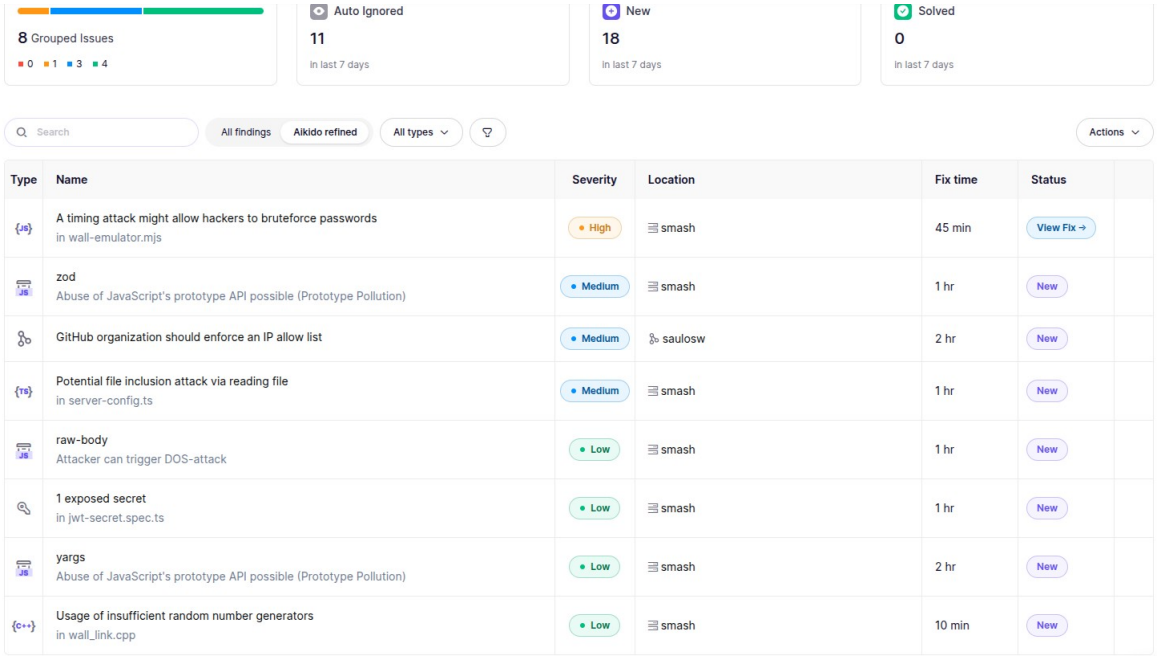
4 VARREDURA DO AIKIDO E CORREÇÃO DOS ACHADOS

4.1 Como a análise foi feita

Em aula, o professor indicou plataformas on-line de análise de repositório e pediu o relatório gerado pela ferramenta. A equipe usou o Aikido Security (https://app.aikido.dev), que reúne em uma só análise o código (SAST), as dependências, os segredos e a configuração da conta no GitHub. A análise é de 31/10/2026 e foi feita sobre saulosw/smash, cópia pública do repositório da disciplina que recebe os mesmos commits.

O relatório gerado está no arquivo Entrega-2-Relatorio-Aikido-Security-Audit-Smash.pdf, que acompanha este plano. A geração do PDF é paga no Aikido, então ele foi salvo a partir da pré-visualização da própria ferramenta. O relatório traz as medidas do projeto para cada item do OWASP Top 10 e de normas como a ISO 27001, o histórico de varreduras e a contagem de achados. O detalhe de cada achado está nas capturas: a Figura 1 mostra o painel, e o Anexo C, cada achado.

Figura 1 - Painel do Aikido com os 8 achados agrupados (captura de 31/10/2026).



O painel agrupou 8 achados: nenhum crítico, 1 alto, 3 médios e 4 baixos. A nota da ferramenta é automática, então cada achado foi conferido no código antes de virar trabalho.

4.2 Conferência dos achados

O Quadro 5 mostra o resultado. A coluna "Aikido" traz a severidade e, quando a captura mostra, a nota de risco. Os números de linha são os da análise, antes das correções.

Quadro 5 - Achados do Aikido e tratamento dado pela equipe.

Achado	Aikido	Análise da equipe	Tratamento	Risco
Ataque de tempo na comparação de senha (código) *scripts/wall-emulator.mjs:139-142*	Alto (80)	Real, mas só no emulador, uma ferramenta de desenvolvimento que não guarda dado real. O firmware já comparava sem parar no primeiro caractere (R02).	Corrigido com *timingSafeEqual*	R29
zod 3.25.76, 4 alertas (dependência) *frontend/package-lock.json*	Médio	Não alcançável. A cópia antiga é do @expo/cli. O aplicativo usa o zod 4.6.7, acima da correção (4.6.0), e é só ele que está no pacote Android.	Justificado. A cópia é interna ao Expo	R11
Sem lista de IPs permitidos no GitHub (conta) *saulosw*	Médio (60)	Não se aplica como está: o recurso exige GitHub Enterprise Cloud, e saulosw é uma conta pessoal. O código é público; o que importa é proteger quem tem acesso de escrita.	Aceito, com autenticação em dois fatores	R30
Possível inclusão de arquivo (código) *backend/src/server-config.ts:39*	Médio (45)	Falso positivo. Os caminhos vêm de *TLS_KEY_PATH* e *TLS_CERT_PATH*, definidos por quem opera o servidor e lidos na subida. Nenhuma requisição chega a esse código.	Sem mudança	-
raw-body 3.0.2 (dependência) *backend/package-lock.json*	Baixo (37)	Não alcançável. A falha exige um *limit* inválido, e a API usa o limite padrão, de 100 KB: um corpo de 150 KB recebeu 413.	Justificado. A versão 4.0.0 exige Node 22 e está fora da faixa do body-parser	R11
1 segredo exposto (segredos) *backend/test/unit/ infrastructure/ jwt-secret.spec.ts:5*	Baixo (27)	Falso positivo. Era um valor inventado para o teste, que não aparece em nenhum outro arquivo do repositório nem no .env local.	Teste passou a gerar o valor na hora	-
yargs 17.7.3 (dependência) *frontend/package-lock.json* e *backend/package-lock.json*	Baixo (24)	Não alcançável. Só ferramentas de linha de comando o usam (TypeORM, Metro, React Native, Jest). A API em execução não o carrega, e ele não está no pacote Android.	Justificado. A versão 18 é só ESM e vem por ferramentas de terceiros	R11
Gerador aleatório insuficiente (código) *backend/firmware/ smash_wall/ wall_link.cpp:204*	Baixo (15)	Falso positivo no contexto. O valor só identifica a inicialização da parede e não protege nada. No núcleo ESP32 3.3.11, o *random()* já usa o gerador de hardware.	Trocado por *esp_random()*, para deixar isso explícito	-

- **Corrigidos (3):** a comparação do emulador (R29), que era real, e dois falsos positivos em que corrigir custava menos que explicar: o valor fixo do teste e o *random()* do firmware.
- **Não alcançáveis (3):** zod, raw-body e yargs ficam registrados no R11, com a prova de que o trecho vulnerável não é usado pelo sistema em execução.
- **Aceito (1):** a lista de IPs do GitHub (R30), que exige um plano pago e não muda a exposição de um repositório público.
- **Falso positivo sem mudança (1):** a leitura dos arquivos do certificado em *server-config.ts*.

4.3 Situação na varredura de 04/11/2026

O Security Audit Report foi gerado em 04/11/2026, sobre a varredura do dia anterior, já com as correções no repositório. Os achados de código corrigidos (a comparação do emulador e o gerador do firmware) não aparecem mais. O segredo do teste continua listado, porque a busca de segredos olha todo o histórico do Git. Também continuam abertos os achados justificados no Quadro 5 e alertas novos de dependências (minimatch, nanoid, picomatch e glob), que entram no R11 e serão conferidos na próxima rodada (seção 6).

O Aikido apontou itens que as outras ferramentas não marcaram: o Semgrep não indicou a comparação do emulador, o gitleaks não indicou o valor do teste, e o npm audit não lista raw-body nem yargs. Por outro lado, três dos oito achados eram falsos positivos, por isso cada um foi conferido no código antes de virar trabalho.

5 MITIGAÇÕES IMPLEMENTADAS NO CÓDIGO

O Quadro 6 liga cada correção ao commit que a implementa. Todos os commits estão no repositório público, e o código entregue é o da tag entrega2-ciberseguranca-aikido, na pasta src/Entrega1.

Quadro 6 - Correções no código e como foram testadas.

Risco	Commit	O que mudou e como foi testado
R01	810a16b e c98a80b	A API serve HTTPS quando recebe um certificado (TLS_KEY_PATH e TLS_CERT_PATH) e deixa de responder por HTTP. Fora do desenvolvimento, o aplicativo não inicia sem um endereço https://. Com um certificado de teste, a API respondeu por HTTPS e recusou HTTP. Falta publicar a API com certificado reconhecido.
R02	3946d66	A parede exige um segredo de pareamento, comparado sem parar no primeiro caractere diferente, e manda os impactos só para o aplicativo pareado. Sem segredo ou com o segredo errado, ela responde "nao autorizado" e fecha a conexão. Falta gravar a placa.
R03	3946d66 e 38529e3	A senha do Wi-Fi e o segredo saíram do config.h para o secrets.h, que fica fora do Git. Sem ele, o firmware não compila ("Falta secrets.h"). A senha antiga é tratada como vazada. Falta gravar a placa com senha nova.
R04	3b5380b	O servidor abre a sessão antes do jogo e devolve um ticket assinado. O resultado só é aceito com o ticket, uma única vez (restrição UNIQUE no banco), com o nível liberado e com duração, batidas e pontos possíveis. Nível bloqueado, resultado repetido, resultado sem ticket, 1 hora inventada e 999.999 pontos foram recusados.
R05	3946d66	A parede atende um aplicativo por vez: um segundo recebe "parede ocupada", e só o pareado manda comandos. Falta gravar a placa.
R06	9a8417b	Limite por IP (10 por minuto, bloqueio de 5 minutos) e por conta (5 a cada 15 minutos, bloqueio de 15 minutos), com resposta 429. Cinco logins errados receberam 401, e o sexto e o sétimo, 429.
R07	3946d66	Comandos com mais de 256 bytes ou acima de 20 por segundo são recusados, e o ping a cada 5 segundos desconecta quem para de responder. A mensagem de 400 bytes foi recusada, e de 30 comandos seguidos, 20 foram aceitos. Falta gravar a placa.
R08	810a16b	CORS desligado por padrão e liberado só para as origens de CORS_ORIGINS. A API escuta em 127.0.0.1 por padrão (HOST). Uma chamada vinda de outro site ficou sem permissão.
R09	da16065	DB_USER e DB_PASSWORD não têm mais valor padrão: sem elas, a API não sobe. Testes de unidade cobrem as duas variáveis.
R10	e8a8476	Token com validade de 12 horas, versão de token por usuário (sair da conta invalida todos os tokens dele) e público próprio. O token usado depois de sair da conta recebeu 401.
R11	b59160e e 3ce31d4	NestJS 10 → 11, multer fixado na 2.5.0 e npm audit fix no aplicativo; o backend ficou sem alertas em 25/10/2026. Os alertas que sobraram estão em ferramentas de desenvolvimento e não entram no pacote Android, o que foi conferido com expo export.
R12	9b5698f	Log em JSON de login, cadastro, logout e requisições recusadas, com o e-mail mascarado e sem senha nem token.
R17	a4cfb7e	npm run demo:accounts cria contas fictícias em demo.smash.test, e npm run demo:clean apaga as contas e os dados delas.

Risco	Commit	O que mudou e como foi testado
R23	4e308b6	helmet e Cache-Control: no-store em todas as respostas. Na nova varredura do OWASP ZAP, os quatro avisos de cabeçalho sumiram.
R24	a27d541	A API não sobe com o segredo JWT de exemplo nem com um segredo de menos de 32 caracteres.
R29	tag entrega2-ciberseguranca-aikido	O emulador da parede passou a comparar o segredo com crypto.timingSafeEqual, como o firmware já fazia (achado do Aikido).

Todas as correções vieram com testes: o backend tem 86 testes e o aplicativo, 117, e todos passam. O script npm run security:test repete 21 ataques contra a API e a parede, e a saída está no Anexo B. O firmware compila para a ESP32. O gitleaks e o Semgrep não acharam nada; o OWASP ZAP e a revisão do histórico deram origem aos riscos R23 e R24.

6 MONITORAMENTO E REVISÃO

O registro segue a rotina do Plano da Entrega 1: revisão semanal na reunião da equipe, antes de qualquer demonstração da parede e a cada marco do projeto. O próximo marco é a apresentação na Semana FECAP de Tecnologia, de 17 a 24/11/2026.

O status de um risco só muda com evidência que pode ser repetida: commit, configuração ou teste. Antes da apresentação, a equipe repete *npm run security:test*, o npm audit e a análise do Aikido e registra o resultado. Riscos novos entram no registro a partir do R31.

6.1 Próximos passos

Depois dos passos que faltam, nenhum risco fica Extremo, e só o R14, que é físico, continua Alto. Antes da apresentação na Semana FECAP de Tecnologia (17 a 24/11/2026), falta:

- **Publicar a API com HTTPS (R01, R22):** servidor com certificado reconhecido, *TRUST_PROXY=true* se ele estiver atrás de um proxy, e *EXPO_PUBLIC_API_URL* com *https://* no aplicativo.
- **Gravar o firmware novo na placa (R02, R03, R05 e R07):** criar o *secrets.h* com senha do Wi-Fi e segredo de pareamento novos, colocar o mesmo segredo em *EXPO_PUBLIC_DEVICE_TOKEN* no aplicativo e rodar *npm run security:test* apontando para a placa, para repetir nela os testes da parede do Anexo B.
- **Preparar o banco do ambiente publicado (R09):** conta própria, senha forte e só as permissões que a aplicação usa.

- **Usar contas fictícias nas demonstrações (R17):** *npm run demo:accounts* antes do evento e *npm run demo:clean* depois.
- **Repetir o npm audit e a análise do Aikido (R11)** antes da apresentação e registrar o resultado.
- **Continuar a validação física (R14)** com a parede montada: um alvo, dois alvos lado a lado e depois os seis.
- **Criar a exclusão da conta pelo jogador (R28).**
- **Proteger o acesso ao repositório (R30):** autenticação em dois fatores nas contas com acesso de escrita.

6.2 Limitações da verificação

- **Placa física:** os testes da parede rodaram no emulador. O firmware novo compila, mas ainda não foi gravado na placa (R02, R03, R05 e R07).
- **Ambiente publicado:** os testes usaram a API e o banco locais, e o HTTPS foi testado com certificado de teste, não com um certificado reconhecido (R01).
- **Aikido:** os 11 itens que a ferramenta ignorou sozinha não foram analisados, e os achados justificados ainda aparecem como abertos no relatório da ferramenta.

APROVAÇÃO

Quadro 7 - Aprovação do documento.

Nome	Cargo	Assinatura	Data
Saulo Pereira de Jesus	Gerente do Projeto	Saulo Pereira de Jesus	06/11/2026
Felipe Vallim Soares	Responsável - Firmware (ESP32)	Felipe Vallim Soares	06/11/2026
Guilhermy Mariano Lisboa Garcia	Responsável - Backend (API e banco)	Guilhermy Mariano Lisboa Garcia	06/11/2026
Gustavo Oliveira Demetrio	Responsável - Aplicativo Mobile	Gustavo Oliveira Demetrio	06/11/2026
Prof. Victor Bruno Alexander Rosetti	Patrocinador do Projeto		

ANEXO A - REGISTRO DE RISCOS ATUALIZADO

O Quadro 8 traz os 19 riscos da Entrega 1 e os onze novos, com a severidade antes e depois das correções.

Quadro 8 - Registro de riscos atualizado (severidade na Entrega 1 e severidade residual).

ID	Risco	Antes	Status	Prob.	Impacto	Depois
R01	A API usa HTTP sem criptografia e o aplicativo envia o token JWT nesse canal.	Extremo	Concluído no código; falta publicar	Baixa	Muito Alto	Médio
R02	O WebSocket da parede usa ws:// e aceita qualquer cliente sem autenticação.	Extremo	Concluído no código; falta gravar a placa	Baixa	Alto	Médio
R03	A senha do SoftAP é previsível e está versionada em repositório público.	Extremo	Concluído no código; falta gravar a placa	Muito Baixa	Alto	Baixo
R04	O resultado da sessão é declarado pelo aplicativo e aceito sem prova de que a partida ocorreu.	Alto	Concluído	Média	Médio	Médio
R05	Qualquer cliente conectado pode trocar o identificador de sessão da parede.	Alto	Concluído no código; falta gravar a placa	Baixa	Médio	Baixo
R06	Login e cadastro não têm limitação de tentativas.	Alto	Concluído	Baixa	Médio	Baixo
R07	O ESP32 não limita clientes, tamanho de mensagens ou frequência de comandos.	Alto	Concluído no código; falta gravar a placa	Baixa	Alto	Médio
R08	A API aceita qualquer origem CORS e escuta em todas as interfaces.	Médio	Concluído	Muito Baixa	Médio	Baixo
R09	O banco usa usuário e senha padrão de desenvolvimento como reserva.	Alto	Concluído	Baixa	Muito Alto	Médio
R10	O JWT dura sete dias e não possui revogação no servidor.	Médio	Concluído	Baixa	Médio	Baixo
R11	As dependências possuem alertas conhecidos no npm audit.	Alto	Concluído	Baixa	Médio	Baixo
R12	Eventos de segurança têm pouca trilha de auditoria.	Médio	Concluído	Baixa	Baixo	Baixo
R13	A integração MQTT prevista não tem controles de segurança definidos.	Alto	Encerrado: MQTT fora do escopo	-	-	-
R14	Isolamento mecânico insuficiente faz a parede registrar o alvo errado.	Alto	Em andamento	Média	Alto	Alto

ID	Risco	Antes	Status	Prob.	Impacto	Depois
R15	Tempo insuficiente para concluir os passos pendentes até a apresentação (17 a 24/11). Reformulado.	Alto	Em andamento	Baixa	Alto	Médio
R16	Alteração de critérios, datas ou formatos das entregas.	Médio	Em andamento	Média	Médio	Médio
R17	Dados pessoais reais usados em testes e demonstrações.	Alto	Concluído	Baixa	Alto	Médio
R18	Falha ou dano da única placa ESP32 disponível.	Médio	Em andamento	Baixa	Alto	Médio
R19	Rede Wi-Fi do local de demonstração indisponível ou instável.	Médio	Encerrado	Muito Baixa	Médio	Baixo
R20	Segredo de pareamento dentro do pacote do aplicativo.	-	Em andamento	Baixa	Médio	Baixo
R21	Versões antigas do aplicativo deixam de salvar sessões.	-	Concluído	Média	Baixo	Médio
R22	Limitação de tentativas atrás de um proxy HTTPS.	-	Concluído	Média	Médio	Médio
R23	API sem cabeçalhos de segurança e com respostas guardáveis em cache.	-	Concluído	Muito Baixa	Baixo	Muito Baixo
R24	API aceitava o segredo JWT de exemplo.	-	Concluído	Muito Baixa	Muito Alto	Médio
R25	Bloqueio de login de outra pessoa pelo limite de tentativas.	-	Aceito	Média	Baixo	Médio
R26	Segredo de pareamento no endereço da conexão, em texto claro.	-	Aceito	Baixa	Médio	Baixo
R27	Cadastro revela se o e-mail já tem conta.	-	Aceito	Média	Baixo	Médio
R28	Sem exclusão da conta pelo próprio jogador (LGPD).	-	Não iniciado	Média	Médio	Médio
R29	O emulador comparava o segredo de pareamento com !==.	-	Concluído	Muito Baixa	Baixo	Muito Baixo
R30	Conta do GitHub sem lista de IPs permitidos.	-	Aceito	Baixa	Médio	Baixo

ANEXO B - TESTES DE SEGURANÇA

As saídas abaixo foram copiadas do terminal nas execuções feitas para este plano. Só foram retiradas as cores, os horários e as linhas repetidas. Os endereços são da máquina de testes (127.0.0.1), e qualquer pessoa pode repetir os testes com os comandos indicados.

Script de testes de segurança

Comando: *npm run security:test*, com a API e o emulador da parede no ar.

```
API http://127.0.0.1:3990 · parede ws://127.0.0.1:8282


[OK]     R06  7 logins com senha errada na mesma conta -> 401, 401, 401, 401, 401, 429, 429
[OK]     R27  6 logins num e-mail que não tem conta -> 401, 401, 401, 401, 401, 429 (igual a uma
conta real)
[OK]     R27  cadastrar um e-mail que já existe -> 409 "Este e-mail já está em uso." (risco
aceito)
[OK]     R10  usar o token depois de sair da conta -> antes 200, saída 204, depois 401
[OK]     R04  começar o nível 5 sem ter passado do 4 -> 400 "Sessão de treino inválida: nível
ainda bloqueado."
[OK]     R04  enviar um resultado legítimo -> 201, 120 XP
[OK]     R04  enviar o mesmo resultado de novo -> 409 "Esta sessão de treino já foi registrada."
[OK]     R04  enviar resultado sem comprovante -> 400 "A sessão precisa ser iniciada antes de ser
registrada."
[OK]     R04  declarar 1 hora de jogo logo depois de começar -> 400 "Sessão de treino inválida:
duração maior que o tempo desde o início."
[OK]     R04  trocar o nível no resultado -> 400 "Sessão de treino inválida: modo ou nível
diferente do iniciado."
[OK]     R04  declarar 999.999 pontos com 6 acertos -> 400 "Sessão de treino inválida: pontuação
acima do máximo possível."
[OK]     R04  usar o comprovante como token de acesso -> 401
[OK]     R08  chamar a API a partir de um site qualquer -> sem permissão
[OK]     R23  conferir os cabeçalhos de segurança -> nosniff=nosniff, x-powered-by=ausente,
cache=no-store
[OK]     R02  conectar à parede sem senha de pareamento -> "nao autorizado", fechada: true
[OK]     R02  conectar com a senha de pareamento errada -> "nao autorizado", fechada: true
[OK]     R02  conectar com a senha certa -> hello
[OK]     R05  conectar um segundo aplicativo com a senha certa -> "parede ocupada", fechada: true
[OK]     R07  mandar uma mensagem de 400 bytes -> "comando grande demais"
[OK]     R07  mandar 30 comandos de uma vez -> 20 aceitos, 10 recusados
[OK]     R05  parear outro aplicativo depois que o primeiro saiu -> hello


21 de 21 testes passaram.
```

ANEXO C - CAPTURAS DO AIKIDO

Cada tela mostra o detalhe de um achado do Quadro 5, como o Aikido o apresentou em 31/10/2026, antes das correções. A própria ferramenta mascara o valor do segredo na Figura 7.

Figura 2 - Ataque de tempo em wall-emulator.mjs.

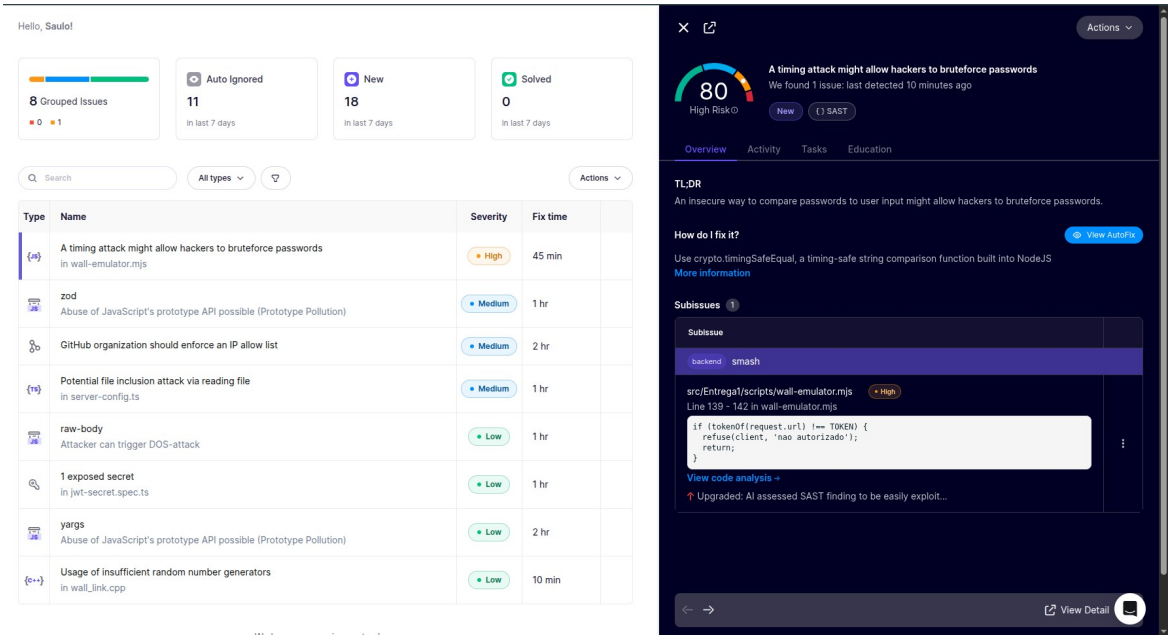


Figura 3 - Alertas do zod.

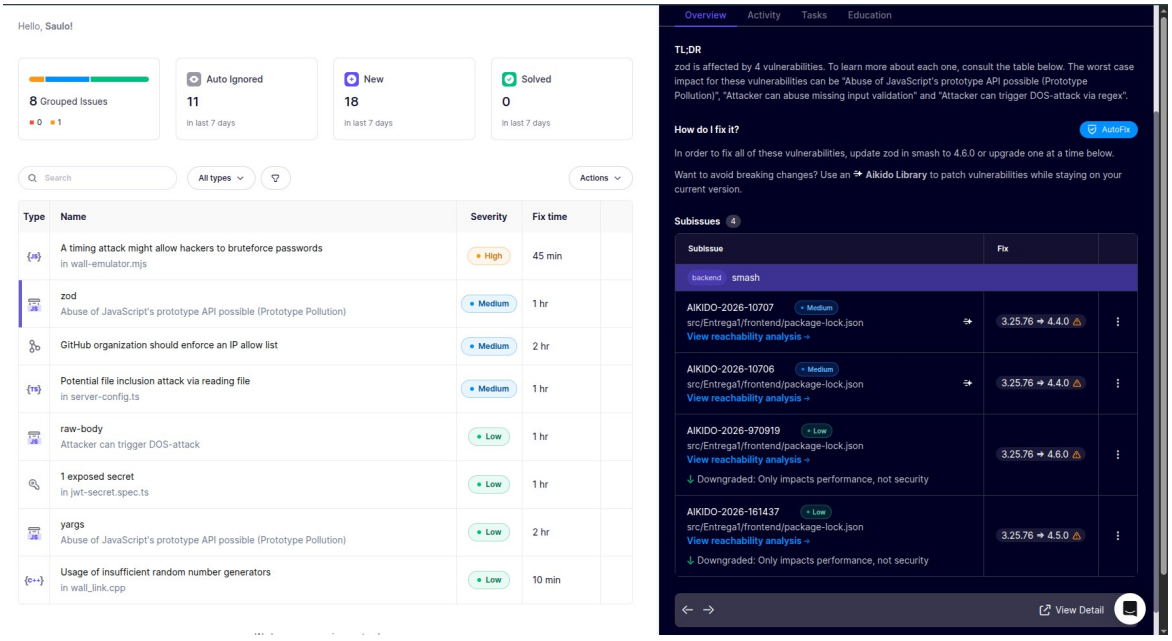


Figura 4 - Lista de IPs permitidos no GitHub.

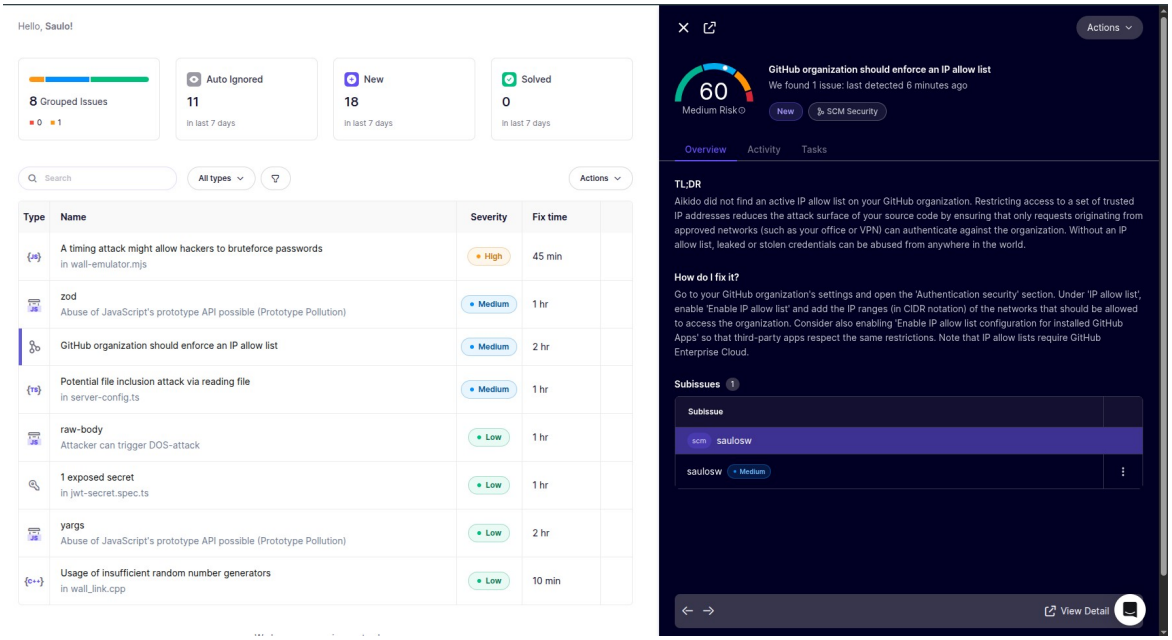


Figura 5 - Possível inclusão de arquivo em server-config.ts.

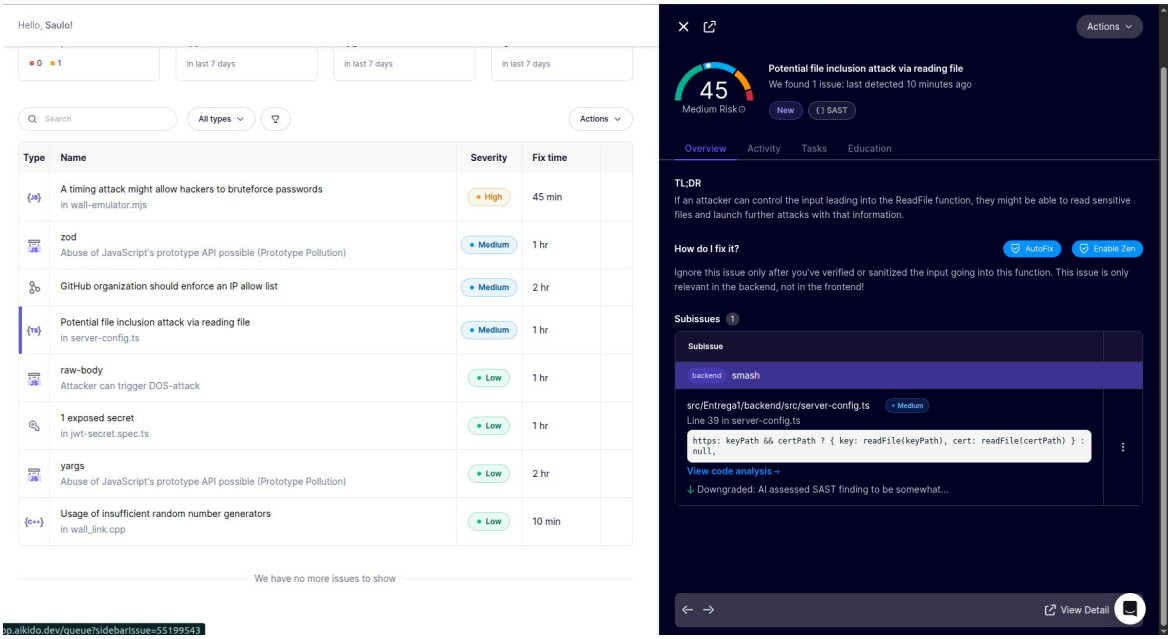


Figura 6 - Alerta do raw-body.

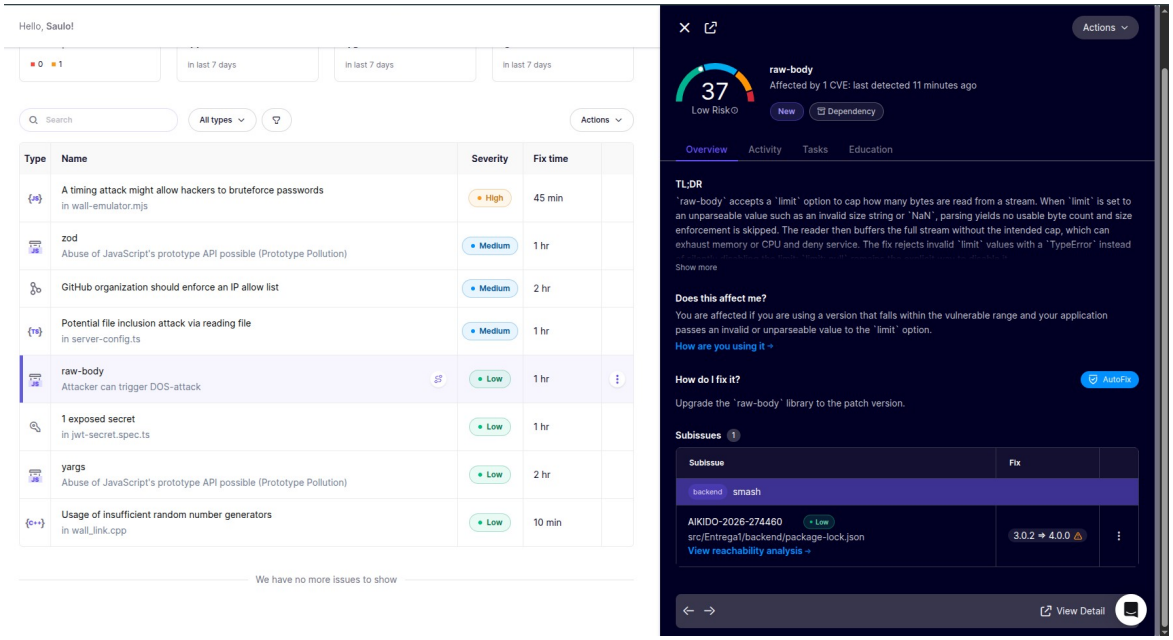


Figura 7 - Segredo em jwt-secret.spec.ts.

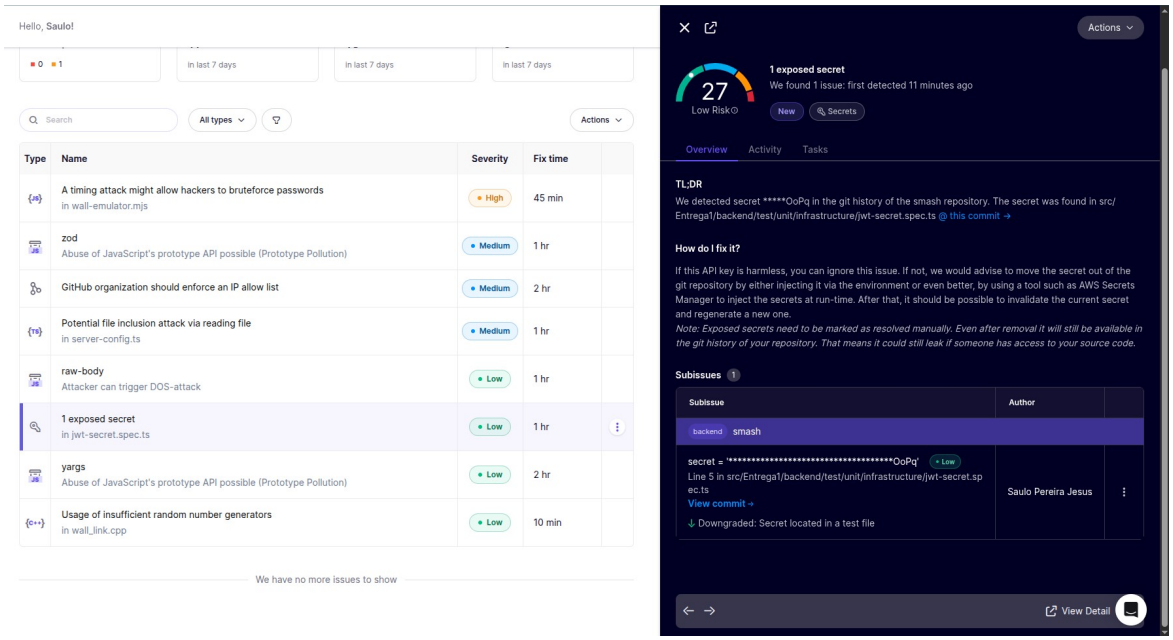


Figura 8 - Alerta do yargs.

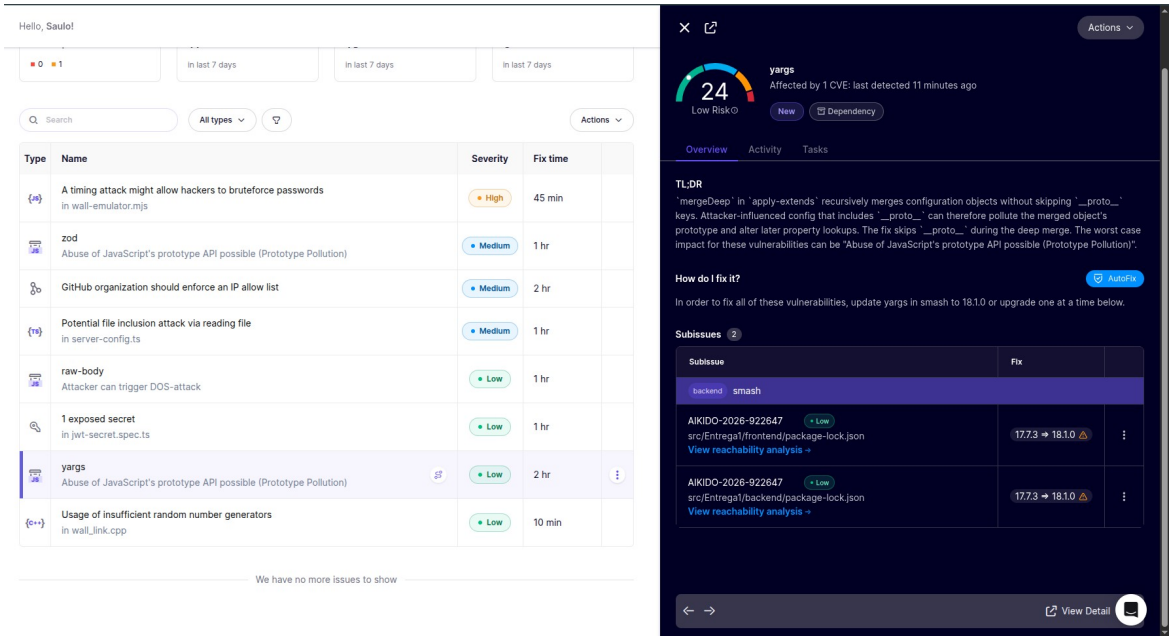


Figura 9 - Gerador aleatório em wall_link.cpp.

